Day 14: K-Nearest Neighbors (KNN)

AI and Machine Learning Course

Sandesh Dhital

Nesfield International College

August 5, 2026

Today's Agenda

What is K-Nearest Neighbors (KNN)?

- A **classification algorithm** based on a simple idea: similar things are close together
- It classifies a new data point based on the **majority vote of its K nearest neighbors**
- **No training phase** — it simply memorizes all training data
- Works for both **binary** (2 classes) and **multiclass** (3+ classes) classification

Intuition

"Tell me who your neighbors are, and I'll tell you who you are."

KNN Algorithm: Step by Step

1. Choose a value for **K** (number of neighbors)
2. For a new point, calculate the **distance** to every training point
3. Select the **K closest** training points
4. Assign the class by **majority voting** among those K neighbors

Example

If K=5 and the 5 nearest neighbors are: Adelie, Adelie, Chinstrap, Adelie, Gentoo
$\rightarrow$ Majority = **Adelie** (3 out of 5) $\rightarrow$ Predict **Adelie**

Euclidean Distance

KNN measures "closeness" using a distance metric. The most common is **Euclidean Distance**:

$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \cdots + (p_n - q_n)^2}$$

- Measures the straight-line distance between two points
- Works with any number of features
- Other options exist (Manhattan, Minkowski) but Euclidean is the default

Why Feature Scaling is Critical for KNN

The Problem

Features with larger ranges dominate the distance calculation.

Example: Two penguins with features [bill_length, body_mass]:

- Penguin A: [39.1, 3750], Penguin B: [46.1, 5200]
- $d = \sqrt{(39.1 - 46.1)^2 + (3750 - 5200)^2} = \sqrt{49 + 2102500} \approx 1450$
- Body mass dominates! Bill length barely matters.

Solution: StandardScaler

$$z = \frac{x - \mu}{\sigma}$$

Puts all features on the same scale (Mean $= 0$, Std $= 1$).

How to Choose K?

K Value	Effect
Small K (e.g., 1)	Sensitive to noise, complex boundaries, can **overfit**
Large K (e.g., 50)	Smooth boundaries, ignores local patterns, can **underfit**
Optimal K	Found by testing multiple values and comparing accuracy

Tips

- Use **odd K** for binary classification to avoid ties
- Common starting point: $K = \sqrt{n}$ where n = number of training samples
- Plot accuracy vs K to find the best value

Binary vs Multiclass Classification

Aspect	Binary (Logistic Reg)	Multiclass (KNN)
Classes	2 (Yes/No)	3+ (Adelie/Chinstrap/Gentoo)
Confusion Matrix	2×2	N×N (e.g., 3×3)
Metrics	Computed directly	Computed per-class, then averaged
Averaging	Not needed	Macro / Micro / Weighted

KNN Advantage

KNN **naturally supports multiclass** classification through majority voting. No special modification needed.

Multiclass Confusion Matrix (3×3)

		Predicted		
		Adelie	Chinstrap	Gentoo
Actual	Adelie	30	1	0
	Chinstrap	1	13	0
	Gentoo	0	0	22

- **Diagonal** = Correct predictions
- **Off-diagonal** = Misclassifications
- Each row shows how samples of one actual class were classified
- Each column shows what was predicted as that class

Multiclass Metric Averaging Strategies

Macro Average

Calculate metric for each class separately, then take the **simple average**. Treats all classes equally — each class matters the same.

Weighted Average

Calculate metric for each class, but give **more weight to larger classes**. Useful when some classes have more samples than others.

Micro Average

Combine all predictions together and compute one overall score. When classes are balanced, this equals accuracy.

Precision and Recall (Multiclass)

Precision (per class)

$$Precision_i = \frac{TP_i}{TP_i + FP_i}$$

Of all samples predicted as class i, how many actually belong to class i?

Recall (per class)

$$Recall_i = \frac{TP_i}{TP_i + FN_i}$$

Of all actual samples of class i, how many did we correctly identify?

F1 Score (Multiclass)

F1 Score (per class)

$$F1_i = 2 \times \frac{Precision_i \times Recall_i}{Precision_i + Recall_i}$$

- Harmonic mean of Precision and Recall for each class
- **Macro F1** is the most commonly reported metric for multiclass problems
- Balances both types of errors into a single score

Tip

Always report the **full classification report** which shows per-class precision, recall, and F1 along with macro and weighted averages.

KNN vs Logistic Regression

Feature	KNN	Logistic Regression
Type	Memorizes data	Learns a formula
Training	No training needed	Trains on data
Boundary	Non-linear, flexible	Linear
Scaling	Required	Recommended
Interpretability	Hard to explain why	Easy (feature weights)
Prediction Speed	Slow	Fast
Multiclass	Works directly	Needs extra setup
Best For	Small datasets, complex patterns	Large datasets, simple patterns

Summary

1. KNN classifies by **majority vote** of K nearest neighbors
2. Uses **Euclidean distance** to find nearest neighbors
3. **Feature scaling is essential** because KNN is distance-based
4. Choose K carefully: too small = overfit, too large = underfit
5. For **multiclass** metrics, use averaging strategies:
   - **Macro**: treats all classes equally
   - **Weighted**: accounts for class size
6. KNN naturally handles multiclass problems without modification
7. Simple, intuitive, but can be slow on large datasets